WEB DEVELOPMENT · 3RD SEMESTER

Week 05 · Block 5a

JavaScript: Values & Functions

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Use `let` / `const` and JavaScript's **dynamic** types
- Always reach for `===` — and explain why `==` is a trap
- Reason about **truthy / falsy** values and handle optional data (`??`, `?.`)
- Treat **functions as values** — store, pass, and return them
- Map it all back to what you already know from **Java**

HTML gave structure. CSS gave it a look.

JavaScript makes it move.

It's the language for the rest of the course — and you already know how to program. Only the rules are new.

PART 1

Values & Variables

You already know this

- You've written **Java**: variables, types, `if` / `else`, loops, methods, classes
- JavaScript has all of it — the **concepts transfer directly**
- What's new is the **rules and quirks** — so that's what we focus on

JavaScript runs in the **browser** (that's what makes pages interactive) — and, as you saw in Block 0, in **Node** on your machine.

Dynamic typing

```
let x = 42;       // a number
x = "hello";      // now a string — no error
x = true;         // now a boolean
```

- **Java**: a variable's type is fixed — `int x = 42;`
- **JavaScript**: a variable holds **any** type, and it can change — **dynamic typing**
- Flexible, but fewer compile-time guarantees → your discipline matters more

let & const

```
const name = "Ada";     // cannot be reassigned
let count = 0;          // can be reassigned
count = count + 1;
```

- `const` **by default** — for values you won't reassign
- `let` only when you truly need to reassign
- **Avoid** `var` — old, confusing scoping; you'll only meet it in legacy code
- Both are **block-scoped** `{ }`, just like Java

The primitive types

- **number** — one type for ints *and* floats (`42`, `3.14`) — no `int` / `double` split
- **string** — `"text"`, `'text'`, or `` `templates` ``
- **boolean** — `true` / `false`
- **null** — a deliberate "nothing"
- **undefined** — "not set yet"

```
const greeting = `Hello, ${name}!`;   // template literal — embed values with ${ }
```

PART 2

Equality — the trap

Two kinds of equal

```
1 == "1"      // true  😱  — types coerced first
1 === "1"     // false     — different types, so not equal
```

- `==` compares **after type coercion** — it converts types, then checks
- `===` compares **value *and* type** — no conversion, no surprises

The coercion rules produce baffling results…

The `==` hall of shame

```
0 == ""              // true
0 == "0"             // true
"" == "0"            // false   ← wait, what?
null == undefined    // true
[] == false          // true
```

- These are *why* `==` is dangerous — the rules aren't consistent
- You will never memorize them, and you shouldn't try

Always use ===

The rule: always use `===` and `!==`. No coercion, no surprises, predictable every time. (One accepted idiom: `x == null` to catch both `null` and `undefined` at once — but when in doubt, use `===`.)

This one habit removes a whole category of bugs. Turn on your linter's `eqeqeq` rule and forget `==` exists.

PART 3

Truthiness & Optionals

Truthy & falsy

In a boolean context (`if`, `&&`, `||`), **every value** is either **truthy** or **falsy**.

The **falsy** values — memorize these six:

```
false     0     ""     null     undefined     NaN
```

Everything else is truthy — including `"0"`, `"false"`, `[]`, and `{}`.

```
if (username) { … }   // runs when username is a non-empty string
```

null vs undefined

- `undefined` — the default "absent": a declared-but-unassigned variable, a missing property, a function that returns nothing
- `null` — an **intentional** "no value" that *you* assign

Both are falsy, both mean "nothing" — `undefined` = *"was never set"*, `null` = *"set to empty on purpose"*.

Defaults & optional access

```
const name = input || "Guest";        // fallback if input is FALSY
const name = input ?? "Guest";        // fallback only if null / undefined
const city = user?.address?.city;     // undefined instead of crashing
```

- `||` — default when the left side is **falsy** *(careful: `0` and `""` trigger it!)*
- `??` — nullish coalescing: default **only** for `null` / `undefined` *(usually what you want)*
- `?.` — optional chaining: safely read a maybe-missing property

PART 4

Functions

Declaring functions

```
function add(a, b) {
  return a + b;
}

const add = (a, b) => a + b;   // arrow function — same idea, compact
```

- A classic `function` declaration, or a compact **arrow function**
- Arrows shine for short functions and callbacks
- No type annotations on parameters — dynamic typing again

Functions are values

```
const greet = function (name) { return `Hi, ${name}`; };

greet;              // the function itself — a value you can hold
greet("Ada");       // calling it → "Hi, Ada"
```

- A function is just a **value** you can store in a variable
- Java has lambdas; in JavaScript this is **everywhere** and foundational
- Because they're values, you can **pass** them and **return** them

Pass a function in — a callback

```
function repeat(n, action) {
  for (let i = 0; i < n; i++) action(i);
}

repeat(3, (i) => console.log("tick", i));
```

- `action` is a **function passed as an argument** — a **callback**
- `repeat` decides *when* to call it
- This is exactly how **array methods** and **events** work (Block 5b & Block 6)

Return a function

```
function multiplier(factor) {
  return (n) => n * factor;
}

const double = multiplier(2);
double(5);   // 10
```

- A function can **build and return** another function
- `double` remembers `factor` even after `multiplier` finished — a peek at **closures** (next block!)

Recap

- JavaScript is **dynamically typed**: `const` / `let` (never `var`), one `number` type
- **Always** `===` — `==` coerces types and lies
- Every value is **truthy / falsy**; know the six falsy values; use `??` and `?.` for optionals
- Functions are **first-class values** — store them, pass them as callbacks, return them
- You already know programming — this was just JavaScript's rules

<EXERCISE> · 45 MIN

Your turn 👉

Prove the quirks to yourself — in code, not on faith.

1. Predict a batch of `==` vs `===` results, then run them; fix a bug caused by `==`
2. Play with **truthy/falsy**, and reach a value safely with `?.`
3. Store a function in a variable, **pass** one as a callback, and **return** one

Starter: `exercises/week05/5a/` — open `README.md`. A `script.js` with `TODO`s runs with `node`.